■ Spinneys

Conversational Shopping Assistant

Week 1 & Week 2 — Deliverables Report

Generated: October 13, 2026 14:45

Phase	Scope	Status
Week 1	DB Schema · Data Quality · Ingestion Pipeline · Embeddings	■ Complete
Week 2	Vector Search · Metadata Filters · FastAPI · Memory Rules	■ Complete

AI Engineering Team · Spinneys Digital & Technology

CONFIDENTIAL — Internal Use Only

1. Executive Summary

This report documents the deliverables produced during **Week 1** and **Week 2** of the Spinneys Conversational Shopping Assistant project. The goal of these two weeks was to lay the full data and retrieval foundation: connecting to the production MySQL database, auditing data quality, building a robust ingestion pipeline, setting up a local ChromaDB vector store with multilingual embeddings, and exposing the retrieval layer via a FastAPI service with metadata-filtering capabilities.

The assistant is designed to understand natural-language shopping queries in **Arabic and English**, resolve product intent, apply contextual memory across conversation turns, and return ranked, filtered product results from the Spinneys product catalogue.

2. Project Timeline

Period	Deliverables
Week 1 — Days 1–2	Environment setup, DB connection, data exploration, schema mapping
Week 1 — Days 3–4	Data quality audit (10 issues identified), ingest pipeline v1
Week 1 — Day 5	ChromaDB integration, first embeddings, VectorStore module
Week 2 — Days 1–2	FastAPI skeleton, /health and /api/search/debug endpoints
Week 2 — Days 3–4	Metadata filters (brand, price, in_stock), query prefix tuning
Week 2 — Day 5	Memory rules design, session state architecture (app/state/)

3. Data Summary

The Spinneys product catalogue is stored in a MySQL 8 database (**spinneys_database**). The following metrics were established during the Week 1 data exploration phase:

Metric	Value
Database	spinneys_database (MySQL 8.0)
Raw products	~124,000 rows in catalog_product
After quality filter	Est. 60,000 – 80,000 clean products
Embedding model	intfloat/multilingual-e5-base (768 dim)
Vector store	ChromaDB — local persistent (cosine similarity)
Languages	English & Arabic (bilingual embedding)
Top-K default	20 results per query

Metric	Value
Retrieval mode	Dense vector search + metadata pre-filtering
API framework	FastAPI 0.111 + Uvicorn
LLM provider	Groq — llama-3.1-8b-instant

Database Schema — Key Tables

Table	Key Columns	Role
catalog_product	id, sku, default_price, default_special_price	Base product record
catalog_productname	product_id, name, language	Multilingual product names
catalog_productattributes	product_id, attributes (JSON), language	JSON array of all product attributes incl. brand
catalog_product_categories	product_id, category_id	Many-to-many product↔category link
catalog_categoryname	category_id, name, language	Multilingual category names
catalog_productstock	sku, source_code, quantity	Per-warehouse stock levels

4. Data Quality Issues (10 Identified)

The following ten data quality issues were identified during the Week 1 audit. Each issue has been addressed either at the SQL query level (pre-fetch) or in the Python **clean_products()** function as a defence-in-depth measure:

ID	Issue	Severity	Pattern	Fix Applied	Impact
DQ-01	**Placeholder Brand**	HIGH	`brand = "Please select a brand."`	Excluded in SQL WHERE + Python filter (INVALID_BRANDS set)	~12% of raw rows
DQ-02	**Empty Brand**	HIGH	`brand = '' (empty string)`	Excluded in SQL WHERE clause and Python defence-in-depth	~5% of raw rows
DQ-03	**Boolean Brand**	MEDIUM	`brand = "false" (JSON boolean leaked to string)`	SQL: NOT IN ('false') — JSON_TABLE cast issue	< 1% of rows
DQ-04	**Zero Price**	HIGH	`default_price = 0 or NULL`	SQL: WHERE p.default_price > 0	~8% of raw rows
DQ-05	**NULL Price**	HIGH	`default_price IS NULL`	Python: pd.to_numeric + dropna fallback	~2% of raw rows
DQ-06	**Test Categories**	MEDIUM	`category name IN ("test", "test1", "test2")`	SQL: LEFT JOIN + LOWER(cn.name) NOT IN (...)	Hundreds of products
DQ-07	**Missing English Name**	MEDIUM	`No row in catalog_productname where language='English'`	SQL: INNER JOIN — products without English name excluded	~3% of products
DQ-08	**Duplicate Rows (Multi-category)**	LOW	`Same product_id appears N times (one per category via LEFT JOIN)`	Python: drop_duplicates(subset=['product_id'], keep best category)	Common — 1 product in 5+ categories
DQ-09	**NULL Stock Entry**	LOW	`No row in catalog_productstock for a given SKU`	SQL: COALESCE(SUM(ps.quantity), 0) → in_stock = 0	~15% of SKUs have no stock record
DQ-10	**NULL / Missing Attributes**	MEDIUM	`catalog_productattributes.attributes IS NULL or missing product_brand_id key`	SQL: INNER JOIN JSON_TABLE → excludes products with NULL attributes	~4% of products

Two-layer defence: SQL-level filters remove the bulk of bad records before data is loaded into Python. The Python clean_products() function applies a second pass (including deduplication) as a safety net.

5. Allowed Metadata Filters

The **VectorStore** class (app/retrieval/vector_store.py) supports the following metadata filters via ChromaDB's where clause. Filters are applied *before* vector ranking, reducing the ANN search space and improving both speed and precision:

Filter Key	Type	Operator	Description	Example Value	Source Column
brand	`str`	`$eq`	Exact brand name match	`brand="Spinneys"`	catalog_productattributes JSON
price_max	`float`	`$lte`	Maximum price (inclusive)	`price_max=50.0`	catalog_product.default_price
in_stock	`bool`	`$eq`	True = in-stock only; False = out-of-stock only	`in_stock=true`	catalog_productstock SUM(quantity)
special_price	`float`	`$lte`	Promotional price ceiling (stored, not filtered yet)	–	catalog_product.default_special_price
category	`str`	`$eq`	Exact English category name (planned Week 3)	–	catalog_categoryname

API Request Example — POST /api/search/debug

```
{ "query": "■■■ ■■■■■ ■■■", "brand": "Spinneys", "price_max": 100.0, "in_stock": true, "top_k": 10 }
```

The SearchFilters dataclass translates these values into a ChromaDB $and compound where clause. When no filters are supplied, the full collection is searched (pure semantic match).

Filter Combination Logic

Scenario	ChromaDB where clause
brand only	`{"brand": {"$eq": "Spinneys"}}`
price_max only	`{"price": {"$lte": 50.0}}`
in_stock only	`{"in_stock": {"$eq": 1}}`
brand + price_max	`{"$and": [{"brand": {"$eq": "X"}}, {"price": {"$lte": 50.0}}]}`
All three filters	`{"$and": [brand_eq, price_lte, in_stock_eq]}`
No filters	where=None (full semantic search)

6. Conversational Memory Rules (6 Rules)

The session memory layer (app/state/) maintains a structured conversation context across turns. The following six rules govern how user intent is accumulated, persisted, and reset:

MR-01 Brand Persistence

Once the user specifies a brand, keep it active for the rest of the session unless they explicitly say 'any brand', 'show all brands', or switch brand.

Example: *User: 'Show me Spinneys cheese' → MR-01 stores brand=Spinneys for next queries*

MR-02 Price Ceiling Accumulation

When the user mentions a maximum price (e.g., 'under 50 EGP'), store it and apply it to every subsequent search until overridden. Never reset on unrelated turns.

Example: *User: 'I need bread under 20 EGP' → price_max=20 persists until 'any price'*

MR-03 Stock Preference Sticky

If the user says 'only available items' or 'in stock', set in_stock=True globally for the session. Reset only if user says 'show everything' or 'including out of stock'.

Example: *User: 'available products only' → in_stock=True applied to all future searches*

MR-04 Quantity & Unit Memory

Remember quantity constraints and units mentioned by the user (e.g., '1 kg', '6 pack', 'family size') and prefer results matching those units.

Example: *User: 'I want 1 litre milk' → prefer products with '1L' or '1000ml' in name*

MR-05 Negative Preference Memory

Track products or brands the user explicitly rejected ('I don't like X', 'not that brand') and exclude them from future results in the session.

Example: *User: 'Not Almarai' → brand=Almarai suppressed for remainder of conversation*

MR-06 Cart / List Accumulation

Maintain a running shopping list across turns. Items the user confirms ('add it', 'yes, that one') are added to the cart state without re-fetching.

Example: *User confirms item → appended to session cart, shown on 'what's in my cart?' query*

Memory rules MR-01 through MR-05 operate on the filter state (SearchFilters dataclass). MR-06 operates on a separate cart accumulator list in the session state. All rules are reset when the user starts a new session or explicitly requests a reset.

7. Technology Stack

Layer	Technology	Version / Detail
Web Framework	FastAPI	≥ 0.111
ASGI Server	Uvicorn	≥ 0.29 (standard)
Data Validation	Pydantic	v2.7+
Vector Store	ChromaDB	≥ 0.5 (persistent, cosine)
Embedding Model	intfloat/multilingual-e5-base	768-dim, bilingual
LLM Provider	Groq	llama-3.1-8b-instant
Database	MySQL 8.0	PyMySQL driver
Config Management	python-dotenv	≥ 1.0.1
Data Processing	Pandas	≥ 2.2

8. System Architecture Overview

```
User Query (AR/EN) ■ ▼ ■■■■■■■■■■■■■■■■ ■ FastAPI ■ POST /api/search/debug ■ (app/main) ■ ■■■■■■■■■■■■■■■■ ■ SearchRequest (query + filters) ▼ ■■■■■■■■■■■■■■■■ ■ VectorStore ■ embed query → ChromaDB.query(where=...) ■ (retrieval) ■ ■■■■■■■■■■■■■■■■ ■ top-K SearchResult objects ▼ ■■■■■■■■■■■■■■■■ ■ Response ■ ranked products + similarity scores ■■■■■■■■■■■■■■■■
```

9. Project File Structure

```
grocery_database/ ■■■ .env ← Runtime config (DB creds, model names) ■■■ .env.example ← Config template for team onboarding ■■■ requirements.txt ← Python dependencies ■■■ app/ ■ ■■■ main.py ← FastAPI app (GET /health, POST /api/search/debug) ■ ■■■ core/ ← Shared config & utilities (Week 3+) ■ ■■■ nlu/ ← Intent & entity extraction (Week 3) ■ ■■■ retrieval/ ■ ■ ■■■ vector_store.py ← VectorStore + SearchFilters + SearchResult ■ ■■■ state/ ← Session memory & cart state (Week 3) ■■■ scripts/ ■ ■■■ ingest.py ← MySQL → ChromaDB ingestion pipeline ■ ■■■ generate_report.py ← This report generator ■■■ data/ ■■■ vectordb/ ← ChromaDB persistent storage (local)
```

10. Next Steps — Week 3 Roadmap

- **NLU Layer:** Build app/nlu/parser.py — LLM-powered intent extraction using Groq (llama-3.1-8b-instant) to parse brand, category, price, quantity, and dietary constraints from free-form Arabic/English queries.
- **Session State:** Implement app/state/session.py — apply all 6 memory rules to persist and update SearchFilters across conversation turns.
- **Conversational API:** Add POST /api/chat — full pipeline: NLU → memory update → vector search → LLM response generation.

- **Hybrid Search:** Combine dense vector search with BM25 keyword search for better recall on exact product names and SKUs.
- **Arabic Query Tuning:** Fine-tune the 'query:' prefix and test Arabic transliteration handling for common Arabised brand names.
- **Evaluation Harness:** Build a test set of 50 benchmark queries (AR + EN) with expected top-3 results; measure Recall@5 and MRR.

End of Report — Spinneys Conversational Shopping Assistant — Week 1 & 2 Deliverables